

Lab 1.2 — Understanding `print()`

□ Objective

By the end of this lab, you will be able to:

- Use `print()` to display strings, numbers, and booleans
 - Print multiple values in a single `print()` call
 - Control the **separator** between values using `sep=`
 - Control the **ending character** using `end=`
 - Understand why `print()` behaviors matter in real scripts
-

□ Background

Why `print()` Is More Powerful Than It Looks

Most beginners think `print()` is just "display text." But Python's `print()` function is a full-featured output tool with optional parameters that give you precise control over how output appears. In IT automation, clean, readable output matters — whether you're printing a server status report, a log summary, or a data table.

Python's Core Data Types (Preview)

You'll go deep on data types in Week 2. For now, here's what Python can print:

Type	Example	Description
<code>str</code> (string)	"Hello"	Text, always in quotes
<code>int</code> (integer)	42	Whole numbers, no quotes
<code>float</code>	3.14	Decimal numbers
<code>bool</code> (boolean)	True or False	Logical values — capital T and F

Real-World Applications

- **Status messages:** `print("Server started successfully.")` — clean operational output
- **Progress indicators:** Using `end=""` to print on the same line as progress updates
- **Report formatting:** Using `sep=` to produce CSV-style or column-aligned output
- **Debugging:** Using `print()` to inspect variable values while developing scripts

□ Lab Overview

Objectives	Master all <code>print()</code> parameters; print all data types
Estimated Time	20–30 minutes
Prerequisites	Lab 1.1 complete; can create and run a <code>.py</code> file

Part C — Exploring `print()` in Depth

Note: This is Part C in the original lab structure, continuing from Part B in Lab 1.1.

Original Code

Create a new file called `print_exploration.py` and type this:

```
# Printing different types of data
print("This is a string")
print(42)
print(3.14)
print(True)

# Printing multiple values in one call
print("Name:", "Alice", "Age:", 30)

# Controlling the separator
print("A", "B", "C", sep="-")

# Controlling the end character (default is newline)
print("Same ", end="")
print("line!")
```

Expected Output

```
This is a string
42
3.14
True
Name: Alice Age: 30
A-B-C
Same line!
```

Code Execution — What Happens Line by Line

Printing Different Data Types

```
print("This is a string")    # Outputs: This is a string
```

```
print(42)           # Outputs: 42
print(3.14)         # Outputs: 3.14
print(True)         # Outputs: True
```

Python's `print()` can handle any data type — it converts everything to a text representation automatically before displaying it.

Printing Multiple Values

```
print("Name:", "Alice", "Age:", 30)
```

Output: Name: Alice Age: 30

When you pass multiple arguments separated by commas, `print()` separates them with a **space by default**. This is the default `sep=" "` behavior.

The sep= Parameter

```
print("A", "B", "C", sep="-")
```

Output: A-B-C

`sep` controls what goes **between** each value. By default it's a space. You can change it to anything:

```
print("2024", "01", "15", sep="/")    # Output: 2024/01/15
print("user", "host", "com", sep="@") # Output: user@host.com
print(1, 2, 3, sep=", ")              # Output: 1, 2, 3
print("A", "B", "C", sep="")          # Output: ABC (no separator)
```

The end= Parameter

```
print("Same ", end="")
print("line!")
```

Output: Same line!

By default, `print()` adds a **newline** (`\n`) at the end of every output — which is why each `print()` starts on a new line. The `end=` parameter overrides this. Setting `end=""` removes the newline, so the next `print()` continues on the same line.

Practical use:

```
# Progress bar style output
print("Loading", end="")
print(".", end="")
print(".", end="")
print(". Done!")
# Output: Loading... Done!
```

Step-by-Step Explanation — Full Parameter Reference

Parameter	Default	What It Does
*objects	(required)	One or more values to print

Parameter	Default	What It Does
sep	" " (space)	String placed between values
end	"\n" (newline)	String placed after the last value
file	sys.stdout	Where to send output (advanced)
flush	False	Force immediate output (advanced)

Full syntax:

```
print(value1, value2, ..., sep=" ", end="\n")
```

□ Key Concepts

Concept	Key Point
print() is a function	It takes arguments inside parentheses
Strings need quotes	Numbers and booleans do not
sep= controls spacing	Default is a single space between values
end= controls line ending	Default is a newline \n
Multiple arguments	Comma-separated values in one print() call
Boolean capitalization	True and False — first letter capitalized, the rest lowercase

⚠ Common Mistakes

1. Lowercase true / false

```
# ❌ WRONG — Python booleans are capitalized
print(true)
print(false)

# ✅ CORRECT
print(True)
print(False)
```

2. Putting Numbers in Quotes When You Want to Do Math

```
# ❌ This prints the string "42", not a number
print("42")

# ✅ This prints the number 42
print(42)
```

This distinction matters when you start doing arithmetic. `"42" + 8` will cause an error — you can't add a string and a number.

3. sep and end Without the =

```
# ❑ WRONG — these are keyword arguments, they need =
print("A", "B", sep "-")

# ❑ CORRECT
print("A", "B", sep="-")
```

4. Forgetting Comma Between Arguments

```
# ❑ WRONG — Python sees "Name:" and "Alice" as one weird expression
print("Name:" "Alice")  # Actually works but concatenates — not intended

# ❑ CORRECT — separate with a comma
print("Name:", "Alice")
```

5. Confusing sep and end

- `sep` goes **between** values in one `print()` call
 - `end` goes **after** all the values at the very end
-

🔧 Additional Practice

Experiment with these in the REPL or a script:

```
# Challenge 1: Print a CSV-style header line
print("Name", "Age", "Department", "Salary", sep=",")

# Challenge 2: Print numbers 1-5 on the same line with spaces
print(1, 2, 3, 4, 5)

# Challenge 3: Print a decorative separator line
print("=" * 30)  # This uses string repetition — preview of Week 2!

# Challenge 4: What does this print? Predict before running.
print("A", "B", sep="---", end="!!!\n")
```

📌 Interview Tips

Q: How do you print multiple values in Python without a space between them?

Use `sep=""` — for example: `print("Hello", "World", sep="")` outputs `HelloWorld`.

Q: How do you print something without a newline at the end?

Use `end=""` — for example: `print("Loading...", end="")` — the next print will appear on the same line.

Q: What data types can `print()` handle?

`print()` can handle any Python object. It calls `str()` on each argument to convert it to a string representation before displaying it.

Real-World Applications

Scenario	Code Pattern
Progress indicator	<code>print("Processing...", end="")</code>
CSV data output	<code>print(name, age, salary, sep=",")</code>
Log timestamps	<code>print("[INFO]", "Server started", sep=" ")</code>
Table formatting	<code>print("Name".ljust(20), "Status".ljust(10))</code>
Status report	<code>print("-" * 40)</code> then print rows

Pro Tips

- **`print()` with no arguments** prints a blank line — useful for spacing in reports.
 - **String * repetition** like `print("-" * 40)` creates separator lines without typing 40 dashes.
 - **`sep="\n"` prints each value on its own line** — useful when printing a list of items.
 - **f-strings** (Week 2+) are the modern, preferred way to format output — `print(f"Hello, {name}!")`.
-

☐ Mini Revision

Quick self-check — answer from memory:

- ☐ How do you print two values separated by a comma instead of a space?
 - ☐ What is the default value of `end=` in `print()`?
 - ☐ How do you make two consecutive `print()` calls appear on the same line?
 - ☐ What does `print(True)` display?
 - ☐ How do you print a blank line?
-

□ Cross References

Previous Topic	Lab 1.1 — Your First Python Program
Next Topic	Practice Problems
Related Concepts	f-strings (Week 2), String methods (Week 2)
Week README	Week 1 Overview

← [Lab 1.1 — First Program](#) | Next: [Practice Problems](#) →